CU7 Performance Assessment — LangganKu (starter)

Fill in your name and the last four digits of your IC, then work through the Tasks of the paper **in this notebook**. First create your own data file in a terminal: `python make_my_data.py <L4>` (the file `subscriber_churn.csv` appears in this folder). Keep the notebook runnable from top to bottom.

In [15]:
```
import warnings; warnings.filterwarnings("ignore")
import mlflow, numpy as np, pandas as pd, requests
import os, sys, json, time, shutil, subprocess
from mlflow import MlflowClient
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline


NAME = "SHAN SWEA WEN JIE"      # e.g. "AISYAH BINTI AHMAD"
L4   = "5277"                # last four digits of your IC, keep the leading zero

FILE   = "subscriber_churn.csv"                   # <- PAPER
ID     = "subscriber_id"                    # <- PAPER  (identifier = NOT a feature
TARGET = "churned"

d = [int(c) for c in L4]                        # 5277 -> d[0]=5 d[1]=2 d[2]=7 d[3]=
SEED      = int(L4)                             # 5277  (NOT 277)
TEST_SIZE = 0.20 if d[3] % 2 == 0 else 0.25     # last digit 7 is odd -> 0.25
DEPTH     = 3 + d[1] % 4                        # 3 + 2 % 4 = 5
C1        = round(0.04 * (d[2] + 1), 2)   # LogisticRegression C   (round() kills 0
C2        = round(C1 * 10, 2)               # EXTRA only: 2nd C value
RANK      = "test_recall"
PORT      = 5100 + int(L4[2:])                  # 5100 + 77 = 5177
EXP       = f"LangganKu_Churn_{L4}"          # <- PAPER (exact spelling)
REG       =f"LangganKu_Churn_{L4}"           # <- PAPER
DB        = f"langganKu_5277.db"
print(dict(SEED=SEED, TEST_SIZE=TEST_SIZE, DEPTH=DEPTH, PORT=PORT, EXP=EXP, REG=REG

EXTRA_4TH_RUN    = False
EXTRA_GAP        = False
EXTRA_TIE        = False
```

```
{'SEED': 5277, 'TEST_SIZE': 0.25, 'DEPTH': 5, 'PORT': 5177, 'EXP': 'LangganKu_Churn_
5277', 'REG': 'LangganKu_Churn_5277', 'DB': 'langganKu_5277.db'}
```

Task 1 — Set-up, split and pipeline

In [2]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.ensemble import RandomForestClassifier

for p in (DB, "mlruns"):                        # fresh store each run (stop the ser
    if os.path.isdir(p): shutil.rmtree(p)
    elif os.path.exists(p): os.remove(p)
mlflow.set_tracking_uri(f"sqlite:///{DB}")
mlflow.set_experiment(EXP)

df = pd.read_csv('subscriber_churn.csv')
print("shape:", df.shape)
print("\ncolumn types:\n", df.dtypes, sep="")
print("\nmissing values:\n", df.isna().sum(), sep="")
print(f"\nclass share of {TARGET}:\n", df[TARGET].value_counts(normalize=True).roun

X = df.drop(columns=[ID, TARGET])               # features = everything except id +
y = df[TARGET]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=TEST_SIZE, rand
cat_cols = X.select_dtypes(exclude="number").columns.tolist()
print("\ntext columns (one-hot):", cat_cols)

def make_pipe(model):                           # ONE pipeline, only the model chang
    pre = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore"), cat_co
    return Pipeline([("prep", pre), ("model", model)])

def score(y_true, y_pred):
    return {"accuracy":  accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred, zero_division=0),
            "recall":    recall_score(y_true, y_pred),
            "f1":        f1_score(y_true, y_pred)}

baseline = make_pipe(LogisticRegression(max_iter=1000)).fit(X_train, y_train)
print("\nbaseline test metrics:", score(y_test, baseline.predict(X_test)))
```

```
2026/10/20 10:33:24 INFO mlflow.store.db.utils: Creating initial MLflow database tab
les...
2026/10/20 10:33:24 INFO mlflow.store.db.utils: Updating database tables
2026/10/20 10:33:26 INFO mlflow.tracking.fluent: Experiment with name 'LangganKu_Chu
rn_5277' does not exist. Creating a new experiment.
```

```
shape: (1200, 10)

column types:
subscriber_id      object
plan               object
avg_hours_week    float64
monthly_fee_rm    float64
contract           object
payment_method     object
main_device        object
support_calls       int64
tenure_months       int64
churned             int64
dtype: object

missing values:
subscriber_id     0
plan              0
avg_hours_week    0
monthly_fee_rm    0
contract          0
payment_method    0
main_device       0
support_calls     0
tenure_months     0
churned           0
dtype: int64

class share of churned:
churned
0    0.788
1    0.212
Name: proportion, dtype: float64

text columns (one-hot): ['plan', 'contract', 'payment_method', 'main_device']

baseline test metrics: {'accuracy': 0.8, 'precision': 0.627906976744186, 'recall':
0.38028169014084506, 'f1': 0.47368421052631576}
```

Task 2 — Track the candidates in ONE experiment

In [12]:
```python
def run_candidate(run_name, model, params, model_type):
    with mlflow.start_run(run_name=run_name) as run:
        pipe = make_pipe(model).fit(X_train, y_train)
        tr = score(y_train, pipe.predict(X_train))
        te = score(y_test,  pipe.predict(X_test))
        mlflow.log_params({"model": type(model).__name__, "test_size": TEST_SIZE, "
        mlflow.set_tag("model_type", model_type)
        mlflow.log_metrics({f"train_{k}": v for k, v in tr.items()})
        mlflow.log_metrics({f"test_{k}": v for k, v in te.items()})
        if EXTRA_GAP:
            mlflow.log_metric("overfit_gap", abs(tr["f1"] - te["f1"]))
        mlflow.sklearn.log_model(pipe, name="model", input_example=X_train.head(3),
    print(run_name, "| train:", {k: round(v, 4) for k, v in tr.items()}, "| test:",
    return run.info.run_id
```

```
# (name, model, params to log, model_type tag)  -- order = order in the paper
candidates = [
    ("logreg-base",     LogisticRegression(max_iter=1000),       {"max_iter": 1000}
    (f"logreg-c{C1:g}", LogisticRegression(C=C1, max_iter=1000), {"C": C1, "max_ite
    (f"logreg-c{C2:g}", LogisticRegression(C=C2, max_iter=1000), {"C": C1, "max_ite
    ((f"rf-d{DEPTH}", RandomForestClassifier(n_estimators=100, max_depth=DEPTH, ran
                    {"n_estimators": 100, "max_depth": DEPTH}, "tree_ensemble"))
]

run_ids = {}
for cname, model, params, mtype in candidates:
    run_ids[f"{cname}-{L4}"] = run_candidate(f"{cname}-{L4}", model, params, mtype)
BASELINE = f"{candidates[0][0]}-{L4}"
```

```
2026/10/20 10:37:04 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle
or cloudpickle format requires exercising caution because these formats rely on Pyth
on's object serialization mechanism, which can execute arbitrary code during deseria
lization. The recommended safe alternative is the 'skops' format. For more informati
on, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
logreg-base-5277 | train: {'accuracy': 0.8544, 'precision': 0.7131, 'recall': 0.475
4, 'f1': 0.5705} | test: {'accuracy': 0.8, 'precision': 0.6279, 'recall': 0.3803, 'f
1': 0.4737}
2026/10/20 10:37:30 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle
or cloudpickle format requires exercising caution because these formats rely on Pyth
on's object serialization mechanism, which can execute arbitrary code during deseria
lization. The recommended safe alternative is the 'skops' format. For more informati
on, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
2026/10/20 10:37:37 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle
or cloudpickle format requires exercising caution because these formats rely on Pyth
on's object serialization mechanism, which can execute arbitrary code during deseria
lization. The recommended safe alternative is the 'skops' format. For more informati
on, see: https://scikit-learn.org/stable/model_persistence.html
logreg-c0.32-5277 | train: {'accuracy': 0.8522, 'precision': 0.7155, 'recall': 0.453
6, 'f1': 0.5552} | test: {'accuracy': 0.8, 'precision': 0.6341, 'recall': 0.3662, 'f
1': 0.4643}
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
logreg-c3.2-5277 | train: {'accuracy': 0.8544, 'precision': 0.7097, 'recall': 0.480
9, 'f1': 0.5733} | test: {'accuracy': 0.8067, 'precision': 0.6383, 'recall': 0.4225,
'f1': 0.5085}
2026/10/20 10:37:44 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle
or cloudpickle format requires exercising caution because these formats rely on Pyth
on's object serialization mechanism, which can execute arbitrary code during deseria
lization. The recommended safe alternative is the 'skops' format. For more informati
on, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
rf-d5-5277 | train: {'accuracy': 0.8489, 'precision': 1.0, 'recall': 0.2568, 'f1':
0.4087} | test: {'accuracy': 0.7967, 'precision': 1.0, 'recall': 0.1408, 'f1': 0.246
9}
```

Task 3 — Choose the best run from code and justify it

In [18]:
```
RANK  = "test_f1"       # paper says which metric (EXTRA rule: odd last digit -> "t
ORDER = "DESC"          # classification scores: higher is better
runs = mlflow.search_runs(experiment_names=[EXP], order_by=[f"metrics.{RANK} {ORDER
cols = ["tags.mlflow.runName", "metrics.train_f1", "metrics.test_f1", "metrics.test
display(runs[cols])

best = runs.iloc[0]
if EXTRA_TIE:                                   # runs within 1% of the best score -
    top  = best[f"metrics.{RANK}"]
    band = runs[(runs[f"metrics.{RANK}"] - top).abs() <= 0.01 * abs(top)]
    best = band.sort_values("metrics.overfit_gap").iloc[0]
BEST_RUN_ID, BEST_NAME = best["run_id"], best["tags.mlflow.runName"]
print("best run:", BEST_NAME, "|", RANK, "=", round(best[f"metrics.{RANK}"], 4))
```

	tags.mlflow.runName	metrics.train_f1	metrics.test_f1	metrics.test_accuracy
0	logreg-c3.2-5277	0.573290	0.508475	0.806667
1	logreg-c3.2-5277	0.573290	0.508475	0.806667
2	logreg-base-5277	0.570492	0.473684	0.800000
3	logreg-base-5277	0.570492	0.473684	0.800000
4	logreg-c0.32-5277	0.555184	0.464286	0.800000
5	logreg-c0.32-5277	0.555184	0.464286	0.800000
6	rf-d5-5277	0.408696	0.246914	0.796667
7	rf-d5-5277	0.408696	0.246914	0.796667

```
best run: logreg-c3.2-5277 | test_f1 = 0.5085
```

Why I chose this run (2–3 sentences):

Task 4 — Register, alias and roll back

In [32]:
```
client = MlflowClient()

v1 = mlflow.register_model(f"runs:/{run_ids[BASELINE]}/model", REG)     # version 1
client.set_registered_model_alias(REG, "champion", v1.version)

v2 = mlflow.register_model(f"runs:/{BEST_RUN_ID}/model", REG)          # version 2
client.set_registered_model_alias(REG, "champion", v2.version)
client.set_model_version_tag(REG, v2.version, "approved_by", NAME)
client.update_model_version(REG, v2.version, description=f"{BEST_NAME}: best {RANK}

client.set_registered_model_alias(REG, "champion", v1.version)          # roll-back
print("after roll-back:    champion -> version", client.get_model_version_by_alias(
client.set_registered_model_alias(REG, "champion", v2.version)          # roll-forw
print("after roll-forward: champion -> version", client.get_model_version_by_alias(

# new rows from the paper: same column ORDER and TYPES as X (ints stay ints, floats
new = pd.DataFrame([
```

```
    {"tenure_months": 2, "support_calls": 4, "monthly_fee_rm": 55.0, "avg_hours_wee
     "plan": "Premium", "payment_method": "Cash voucher", "contract": "Monthly", "m
    {"tenure_months": 36, "support_calls": 0, "monthly_fee_rm": 25.0, "avg_hours_we
     "plan": "Basic", "payment_method": "Card", "contract": "Yearly", "main_device"
    {"tenure_months": 14, "support_calls": 2, "monthly_fee_rm": 40.0, "avg_hours_we
     "plan": "Standard", "payment_method": "E-wallet", "contract": "6-month", "main
])


champion = mlflow.pyfunc.load_model(f"models:/{REG}@champion")
print("predicted class (0 = stays, 1 = churns):", champion.predict(new).tolist())
```

```
Registered model 'LangganKu_Churn_5277' already exists. Creating a new version of th
is model...
2026/10/20 11:14:37 WARNING mlflow.tracking._model_registry.fluent: Run with id 71a3
6b3994514cd0bf0e91ea779ad5bd has no artifacts at artifact path 'model', registering
model based on models:/m-bdce66c9c2244dcfbd10129b9acee05b instead
Created version '21' of model 'LangganKu_Churn_5277'.
Registered model 'LangganKu_Churn_5277' already exists. Creating a new version of th
is model...
2026/10/20 11:14:37 WARNING mlflow.tracking._model_registry.fluent: Run with id 587f
9842212e46f9ae5a1878017be427 has no artifacts at artifact path 'model', registering
model based on models:/m-64ac8deeaa844db7925fe56be6ac3999 instead
Created version '22' of model 'LangganKu_Churn_5277'.
after roll-back:    champion -> version 21
after roll-forward: champion -> version 22
predicted class (0 = stays, 1 = churns): [1, 0, 0]
```

Task 5 — Serve the champion and call it

In [33]:

```
# Terminal way (PowerShell, in this folder, cu7env active):
#   $env:MLFLOW_TRACKING_URI = "sqlite:///pinjampantas_5277.db"
#   mlflow models serve -m "models:/pinjampantas_default_5277@champion" -p 5177 --e
URL = f"http://127.0.0.1:{PORT}"
env = dict(os.environ, MLFLOW_TRACKING_URI="sqlite:///" + os.path.abspath(DB).repla
           __pyfunc_model_path__=f"models:/{REG}@champion")
server = subprocess.Popen([sys.executable, "-m", "uvicorn", "--host", "127.0.0.1",
                           "mlflow.pyfunc.scoring_server.app:app"], env=env,
                          stdout=open("server.log", "w"), stderr=subprocess.STDOUT)
for _ in range(120):
    try:
        if requests.get(URL + "/health", timeout=1).status_code == 200: break
    except requests.ConnectionError:
        time.sleep(1)
print("health:", requests.get(URL + "/health").status_code)
```

```
health: 200
```

In [35]:

```
def predict_via_api(df):
    payload = {"dataframe_split": json.loads(df.to_json(orient="split", index=False
    r = requests.post(URL + "/invocations", json=payload)
    return r.status_code, (r.json()["predictions"] if r.status_code == 200 else r.t

status, preds = predict_via_api(new)
print("good request:", status, preds)
```

```
status, err = predict_via_api(new.drop(columns=["support_calls"]))    # <- PAPER: c
print("bad request :", status, err[:200])
```

```
good request: 200 [1, 0, 0]
bad request : 400 {"error_code": "INVALID_PARAMETER_VALUE", "message": "Failed to pr
edict data '   tenure_months  monthly_fee_rm  ...  contract main_device\n0
2            55.0  ...   Monthly       Phone\n
```

In [36]:
```
# Stop the server at the very end (after check_my_work.py --serve). Set True and ru
STOP = False
if STOP:
    subprocess.run(["taskkill", "/F", "/T", "/PID", str(server.pid)]) if os.name ==
```

In [ ]:
```

```